

Docker Networks: Complete Guide

Introduction, Syntax, ELI5 & ELI10 Explanations

Generated: April 12, 2026

Table of Contents

1. What Are Docker Networks?
2. Why Do We Need Networks?
3. ELI5 Explanation (Simple Version)
4. ELI10 Explanation (Detailed Version)
5. Docker Network Types
6. Network Syntax & Commands
7. Network Concepts (IP, CIDR, Gateway)
8. Real-World Examples
9. Best Practices
10. Your Art of Living Setup

1. What Are Docker Networks?

Docker Networks are isolated network environments that allow containers to communicate with each other and the outside world. Think of them as virtual networks that separate traffic, allowing containers to be grouped logically.

Simple Definition:

A Docker Network = A virtual network where containers can connect, communicate, and be isolated from other networks.

2. Why Do We Need Networks?

Without networks, containers would be isolated and couldn't communicate. Networks solve several problems:

Problem	Solution	Example
Containers isolated	Networks connect them	App container reaches database container
No service discovery	Docker DNS resolves names	Use 'postgres' instead of IP address
Port conflicts	Networks isolate ports	Multiple services on port 3000
Hard to manage	Group containers logically	Separate prod/dev networks

3. ELI5 Explanation (Explain Like I'm 5)

The Classroom Analogy

Imagine a classroom with desks (containers): WITHOUT NETWORKS: All desks are in separate rooms. They can't talk to each other. Tom (container 1) in Room A Sarah (container 2) in Room B They can't share anything or communicate. **WITH NETWORKS:** All desks in the SAME classroom. They can talk and share! Network = The classroom Tom and Sarah can see each other and communicate. **NETWORK TYPE = Different classrooms with different rules:** Bridge Network = Regular classroom (default) Host Network = No classroom (direct access to school) Overlay Network = Virtual classroom spanning multiple buildings **KEY POINT:** Networks = Places where containers can talk to each other!

4. ELI10 Explanation (Detailed Version)

How Docker Networks Work

1. CONTAINER = A process running in isolation ■■ Has its own filesystem ■■ Has its own network interfaces (eth0, eth1...) ■■ Can have multiple network interfaces! **2. NETWORK = Virtual switch (like a network switch in a data center)** ■■ Containers connect to it ■■ Containers can talk to each other ■■ Docker manages routing between them **3. DOCKER DNS = Automatic name resolution** ■■ When you ping 'postgres', Docker DNS resolves to its IP ■■ No need to hardcode IPs ■■ Works automatically on user-defined networks **4. NETWORK ISOLATION = Containers on different networks can't reach each other** ■■ Network A: Container 1, Container 2 ■■ Network B: Container 3, Container 4 ■■ Container 1 can't reach Container 3 (different networks) **5. MULTI-NETWORK CONTAINERS = A container can be on multiple networks** ■■ Container can have eth0 (on Network A) ■■ Container can have eth1 (on Network B) ■■ Container can talk to both networks!

5. Docker Network Types

Type	Scope	Use Case	Default?	Example
Bridge	Single host	Local containers	YES	App + DB on same host
Host	Single host	No isolation (performance)	NO	High-speed services
Overlay	Multi-host (Swarm)	Distributed services	NO	Microservices across servers
Macvlan	Single host	Physical network	NO	Direct MAC address
None	Single host	No networking	NO	Isolated (no network)

6. Network Syntax & Commands

Create a Network

`docker network create [OPTIONS] NETWORK_NAME` Examples: `docker network create my-app`
`docker network create --subnet 172.20.0.0/16 prod-net` `docker network create --driver overlay swarm-net`

List Networks

`docker network ls` Output: NETWORK ID NAME DRIVER SCOPE a1b2c3d4e5f6 bridge bridge
local f1e2d3c4b5a6 my-app bridge local

Inspect Network

`docker network inspect my-app` Shows: ■■ Network name and ID ■■ Driver type ■■ Subnet (CIDR) ■■ Gateway ■■ Connected containers and their IPs

Connect Container to Network

`docker network connect NETWORK_NAME CONTAINER_NAME` Example: `docker network connect my-app postgres-db` Result: Container gets new network interface on my-app

Disconnect Container from Network

`docker network disconnect NETWORK_NAME CONTAINER_NAME` Example: `docker network disconnect my-app postgres-db` Result: Container loses that network interface

Remove Network

`docker network rm NETWORK_NAME` Note: Can't remove if containers are connected!

7. Network Concepts (IP, CIDR, Gateway)

IP Address

A unique identifier for a container on a network. Example: 172.18.0.2 Parts: 172.18 = Network address (fixed) .0.2 = Host address (unique per container) Properties: ■■ Assigned automatically (or can be set manually) ■■ Might change if container restarts ■■ Use hostname (container name) instead!

CIDR Notation

Compact way to represent IP networks. Format: IP/PREFIX Example: 172.18.0.0/24 172.18.0.0 = Network address /24 = First 24 bits are fixed, last 8 bits are variable = 256 total addresses (172.18.0.0 to 172.18.0.255) Common Sizes: /32 = 1 IP /24 = 256 IPs (254 usable) /16 = 65,536 IPs /8 = 16 million IPs

Gateway

The router that connects your network to other networks. Example: 172.18.0.1 In Docker: ■■ First IP in the subnet (usually .1) ■■ Routes traffic in/out of the network ■■ Assigned automatically

8. Real-World Examples

Example 1: Simple Web App (Bridge Network)

```
docker network create web-app docker run -d --name postgres --network web-app \ -e POSTGRES_PASSWORD=secret postgres:15 docker run -d --name api --network web-app \ -p 3000:3000 your-api-image Result: ■■ Both on 'web-app' network ■■ API can reach postgres by name: postgres://postgres:5432 ■■ External users reach API via port 3000
```

Example 2: Multi-Network Service (Your Setup)

```
docker network create pf-prod docker network create flm-prod docker run -d --name nginx --network pf-prod nginx:latest docker network connect flm-prod nginx docker run -d --name rx-service --network pf-prod your-app docker run -d --name flm-service --network flm-prod your-app Result: ■■ nginx connects pf-prod + flm-prod ■■ nginx reaches rx-service via pf-prod network ■■ nginx reaches flm-service via flm-prod network ■■ rx-service and flm-service can't reach each other
```

Example 3: Custom Subnet

```
docker network create --subnet 10.0.0.0/24 custom-net docker run -d --name app --network custom-net \ --ip 10.0.0.10 your-app Result: ■■ Network uses 10.0.0.0/24 (256 IPs) ■■ app gets fixed IP: 10.0.0.10 ■■ Other containers get IPs in 10.0.0.x range
```

9. Best Practices

■ **DO:** ■■ Use user-defined networks (not default bridge) ■■ Name containers clearly ■■ Use container names instead of hardcoding IPs ■■ Use separate networks for prod/dev/staging ■■ Document which networks your services use ■■ Use meaningful network names (prod-net, staging-net)

■ **DON'T:** ■■ Use default bridge network in production ■■ Hardcode IP addresses ■■ Put all services on one network ■■ Forget to document network topology ■■ Use auto-assigned IPs for permanent services

10. Your Art of Living Setup

Current Architecture:

1 EC2 instance with: ■■ 27 PM2 services ■■ PostgreSQL ■■ Nginx reverse proxy ■■ All need to communicate

Recommended Network Setup:

```
docker network create pf-prod --subnet 172.18.0.0/24 # Connect all services
docker run -d --name postgres-tptm --network pf-prod \ -e POSTGRES_PASSWORD=secret
postgres:15 docker run -d --name nginx --network pf-prod nginx:latest
docker run -d --name rx-service --network pf-prod your-app
docker run -d --name bx-fe --network pf-prod your-app # ... add 25 more services ...
Result: ■■ All 29 services on pf-prod network ■■ Services reach each other by name (no IP hardcoding)
■■ Nginx can proxy to any service ■■ Easy to add/remove services ■■ Clean, manageable architecture
```

Check Network Status:

```
docker network inspect pf-prod This shows: ■■ All connected containers ■■ Their IPs (172.18.0.2, 172.18.0.3...) ■■ Subnet (172.18.0.0/24) ■■ Gateway (172.18.0.1)
```